

Technical Note — EV + Developer Engine (5-Year Default)

1) Background & Purpose

This engine distributes project probability across candidate FID years, computes expected values by year, and evaluates developer economics (DevCost, MOIC, k^* , IRR). It is a post-scoring tool, not a re-fit of Bayes or Cox models.

2) Inputs

- Required: `blended_prob` (or `p_bayes/p_cox`), `project_cost`, `years_remaining`, `urgency_scale_(0-1)`
- Optional: `imputed_cost`, `start_year`, `end_year`

3) Methodology

EV Allocation:

Candidate years = `BASE_YEAR+1 .. BASE_YEAR+horizon`. Annual probabilities allocated with softmax weighting (`alpha0`, `alpha1`) using `urgency` and `years_remaining`. EV per year = $P(\text{year}) \times \text{project_cost}$.

Developer Engine:

Cost basis selected from `project_cost` → `imputed_cost` → `value_proxy`. $\text{DevCost} = \text{dev_cost_pct} \times \text{cost_basis}$. Expected FID year = weighted average over candidate years. PV sums computed using S-curve spend paths (S, even, front, back). Metrics include `k_star`, `PV_MOIC_k`, `IRR_k`, `k_star_adj`.

4) Outputs

- `Annual_p_YYYY`, `EV_YYYY` columns
- `EV_cum`, `EV_disc`
- Developer metrics: `DevCost`, `E_FID_year`, `PV_MOIC_k`, `k_star`, `IRR_k`, `k_star_adj`

5) Replication

- Generate input with MPI Risk Engine
- Run EV + Developer Engine with chosen knobs
- Archive inputs/outputs
- Record command/config for audit

6) QA Checks

- $\sum \text{annual_p} \approx \text{total probability}$
- $\text{EV_disc} \leq \text{EV_cum}$ when discount > 0
- S-curve weights sum to 1

- IRR_k finite and reasonable

7) Limitations

Stylized, counterfactual parameters; not calibrated to real projects. Results show marginal and distributional effects, not forecasts.